Coeficiente de Variação com OpenMP

Exercício Final – Programação Paralela e Distribuída – IESB – 2026/2

Aluno: Felipe Oliveira Alves

1. Objetivo e metodologia

Foi implementado em C++ (arquivo **desafioopenmp.cpp**, versão 1.3) o cálculo do coeficiente de variação (CV) da estatura e do peso de 1000 pessoas, com os dados agrupados em intervalos de classe (8 cm para a estatura e 4 kg para o peso) e paralelizado com OpenMP. O programa lê um arquivo texto com uma pessoa por linha (estatura e peso separados por espaço) e usa as fórmulas do enunciado: média ponderada pelos pontos médios das classes, desvio padrão populacional e CV = (desvio / média) × 100%.

Dados de entrada. Como o conjunto de dados não acompanhava o enunciado, foi gerado um arquivo sintético com 1000 pessoas (distribuição normal, semente fixa 42; estatura com média 170 cm e peso com média 70 kg), pelo próprio programa (`./desafioopenmp --gerar dados.txt`). Os valores de CV abaixo valem para esse arquivo.

Classes. O limite inferior da primeira classe é o mínimo dos dados arredondado para baixo; o número de classes é parte inteira de [(máx - limite inferior) / h] + 1, e o ponto médio da classe i é o limite inferior + (i + 0,5)·h.

2. O que foi paralelizado e por quê

Etapa	Diretiva OpenMP	Justificativa
Mínimo e máximo dos dados	parallel for + reduction(min, max)	Percorre os N valores; cada thread analisa um pedaço e o OpenMP combina os resultados.
Frequências por classe (histograma)	parallel for + reduction(+ : f[:k])	É o laço mais pesado (N iterações). Cada thread acumula em um vetor privado e eles são somados no fim, sem condição de corrida e sem critical.
Soma ponderada (média)	parallel for + reduction(+)	Laço sobre as k classes. Paralelizado conforme o enunciado, mas com poucas iterações (8 e 17) o ganho é desprezível.
Soma dos desvios quadráticos (desvio padrão)	parallel for + reduction(+)	Mesmo caso da média.
Leitura do arquivo	(sequencial)	É E/S de disco, que não se beneficia de threads.

Limiar de paralelização. Para N < 100.000, `calcular_par()` chama diretamente a versão sequencial. Em uma versão anterior, sem esse desvio, a execução com 1000 pessoas ficou cerca de 20 vezes mais lenta que a sequencial (speedup de 0,05x), porque o custo de entrar nas regiões paralelas era maior que o trabalho útil. A cada execução o programa também confere que as versões sequencial e paralela produzem o mesmo CV (diferença máxima observada: 8,9e-16, apenas ordem de soma em ponto flutuante).

3. Resultados: coeficiente de variação

Variável	Classe	Média	Desvio padrão	CV
Estatura	8 cm	170,3440 cm	9,4425 cm	5,5432 %
Peso	4 kg	70,4240 kg	12,4147 kg	17,6285 %

4. Análise de desempenho

Ambiente: AMD Ryzen 7 9800X3D (8 núcleos físicos, 16 threads com SMT, 96 MB de cache L3), Windows 11 25H2, g++ com `-O2 -fopenmp -std=c++17`. **Método:** o tempo mede o cálculo completo (estatura + peso), sem a leitura do arquivo. Cada ponto é a mediana de 7 blocos de execuções, após uma execução de aquecimento. Para testar volumes maiores, os 1000 registros foram replicados (opção `--replicar`); isso não altera o CV, apenas o volume de trabalho. Speedup = tempo sequencial / tempo paralelo; eficiência = speedup / nº de threads.

N = 1.000 – tempo sequencial: 0,000002 s

Threads	Tempo (s)	Speedup	Eficiência
1	0,000002	0,98x	98,0%
2	0,000002	0,98x	49,0%
4	0,000002	1,00x	25,0%
8	0,000002	0,98x	12,3%
16	0,000002	0,98x	6,1%

N = 10.000.000 – tempo sequencial: 0,024500 s

Threads	Tempo (s)	Speedup	Eficiência
1	0,024450	1,00x	100,2%
2	0,012550	1,95x	97,6%
4	0,006650	3,68x	92,1%
8	0,005100	4,80x	60,0%
16	0,004650	5,27x	32,9%

N = 50.000.000 – tempo sequencial: 0,122250 s

Threads	Tempo (s)	Speedup	Eficiência
1	0,123000	0,99x	99,4%
2	0,097750	1,25x	62,5%
4	0,051500	2,37x	59,3%
8	0,034500	3,54x	44,3%
16	0,032750	3,73x	23,3%

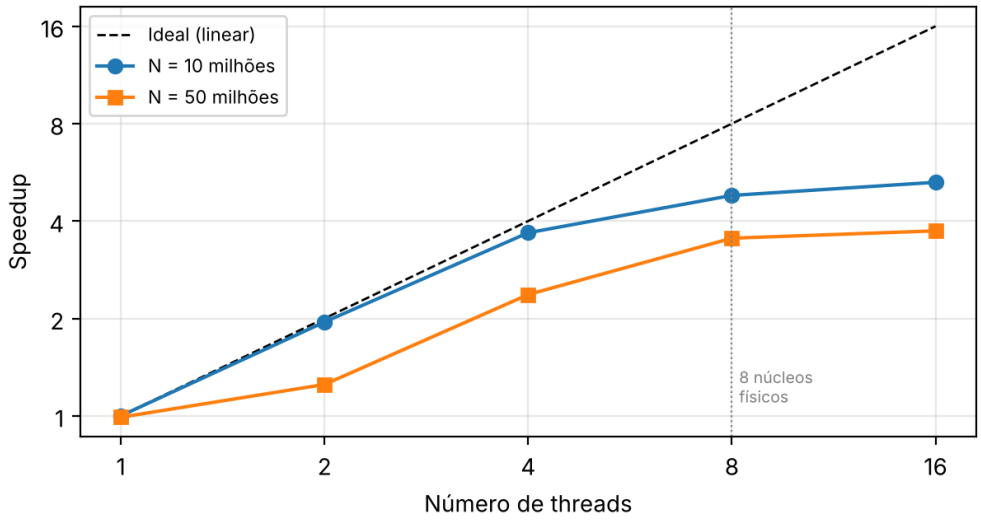


Figura 1 – Speedup medido versus número de threads (escala log-log).

5. Discussão

- **1000 pessoas:** o speedup é cerca de 1,0x em todas as configurações porque, abaixo do limiar, o programa executa o código sequencial. O volume (cerca de 2 microssegundos) é pequeno demais para compensar qualquer paralelismo.
- **10 milhões:** escalonamento quase linear até 4 threads (eficiência acima de 92%) e speedup máximo de 5,27x com 16 threads.
- **50 milhões:** o speedup máximo cai para 3,73x. Com 400 MB por variável, os dados não cabem no cache L3 (96 MB) e o programa passa a depender da memória RAM.
- **Platô a partir de 8 threads:** a máquina tem 8 núcleos físicos; as threads 9 a 16 são SMT, que dividem os mesmos núcleos e quase não acrescentam ganho (4,80x para 5,27x e 3,54x para 3,73x).
- **Indício de limite de memória:** dividindo os bytes lidos por chamada (4 passadas sobre os vetores) pelo tempo, obtém-se cerca de 13 GB/s na versão sequencial e cerca de 46 a 49 GB/s com 8 a 16 threads para 50 milhões. Para 10 milhões, o valor chega a cerca de 69 GB/s, mais alto, o que é coerente com os dados caberem em boa parte no cache L3. São estimativas feitas a partir dos tempos, não medições diretas de largura de banda.
- **Ressalva:** com 2 threads e 50 milhões o speedup foi de apenas 1,25x (1,95x com 10 milhões), valor que não explico apenas pela análise acima e que pode refletir variação de medição ou de frequência do processador.

6. Conclusão

O CV obtido foi de **5,54%** para a estatura e de **17,63%** para o peso; as versões sequencial e paralela produzem resultados equivalentes. A paralelização com reduções do OpenMP trouxe ganho real somente quando o volume de dados é grande (até 5,27x com 10 milhões de registros e 3,73x com 50 milhões), e foi limitada pelos 8 núcleos físicos e, para volumes que excedem o cache, provavelmente pela largura de banda da memória. Para os 1000 registros do enunciado, o tempo total é de poucos microssegundos e a versão sequencial é a mais adequada.

Apêndice A – Distribuições de frequência (N = 1000)

Classe (cm)	Ponto médio	Freq.
[140, 148)	144	6
[148, 156)	152	51
[156, 164)	160	186
[164, 172)	168	339
[172, 180)	176	271
[180, 188)	184	116
[188, 196)	192	28
[196, 204)	200	3

Classe (kg)	Ponto médio	Freq.
[40, 44)	42	17
[44, 48)	46	22
[48, 52)	50	36
[52, 56)	54	59
[56, 60)	58	82
[60, 64)	62	80
[64, 68)	66	114
[68, 72)	70	131
[72, 76)	74	127
[76, 80)	78	108
[80, 84)	82	93
[84, 88)	86	46
[88, 92)	90	47
[92, 96)	94	21
[96, 100)	98	11
[100, 104)	102	5
[104, 108)	106	1